Devoir surveillé d'informatique

Consignes

- On pourra toujours librement utiliser une fonction demandée à une question précédente même si cette question n'a pas été traitée.
- Veillez à présenter vos idées et vos réponses partielles même si vous ne trouvez pas la solution complète à une question.
- La clarté et la lisibilité de la rédaction et des programmes sont des éléments de notation.

❑ Exercice 1 : *Anagrammes*

Deux mots *de même longueur* sont anagrammes l'un de l'autre lorsque l'un est formé en réarrangeant les lettres de l'autre. Par exemple :

— *niche* et *chien* sont des anagrammes.

— *epele* et *pelle* ne sont pas des anagrammes. En effet, bien qu'ils soient formés avec les mêmes lettres, la lettre *l* ne figure qu'à un seul exemplaire dans *epele* et il en faut deux pour écrire *pelle*.

Le but de l'exercice est d'écrire une fonction `anagrammes` qui prend en argument deux chaînes de caractères et qui renvoie `True` si ces deux chaînes sont des anagrammes et `False` sinon.

■ Partie I : Sans utilisation de dictionnaires

Q1– Écrire une fonction `compte_occ` qui prend en argument une chaîne de caractères `chaine` et un caractère `car` et renvoie le nombre d'apparitions de `car` dans `chaine`.

Q2– Montrer à l'aide d'un test approprié que la fonction `anagramme_bug` ci-dessous n'est pas correcte, c'est-à-dire qu'elle peut renvoyer `True` alors que les deux chaînes passées en argument ne sont pas des anagrammes.

```
def anagramme_bug(chaine1, chaine2):
    '''Renvoie True ssi chaine1 et chaine2 sont des anagrammes'''
    for c in chaine1:
        if compte_occ(chaine1,c)!=compte_occ(chaine2, c):
            return False
    return True
```

Q3– Corriger la fonction précédente afin de la rendre conforme à sa spécification.

■ Partie II : Avec les dictionnaires

On souhaite écrire une fonction `cree_dico` qui prend en argument une chaîne de caractères et renvoie un dictionnaire dont les clés sont les caractères composant la chaîne et les valeurs leur nombre d'apparitions.
Par exemple, `cree_dico("epele")` renvoie le dictionnaire `{'e': 3, 'p': 1, 'l': 1 }`. En effet, dans le mot `'epele'`, `'e'` apparaît à trois reprises et `'l'` et `'p'` chacun une fois.

Q4– Expliquer brièvement pourquoi la solution proposée ci-dessous (qui utilise la fonction `compte_occ` de la partie précédente) est de complexité quadratique en fonction de la taille n de la chaîne fournie en argument.

```
def cree_dico_quadratique(chaine):
    dico = {}
    for c in chaine:
        if c not in dico:
            dico[c] = compte_occ(chaine, c)
    return dico
```

Q5– Écrire une version de complexité linéaire de la fonction `cree_dico` puis expliquer brièvement pour quelle raison on obtient cette meilleure complexité.

Q6– Écrire une fonction `egaux` qui prend en argument deux dictionnaires et renvoie `True` si ces deux dictionnaires sont égaux (c'est-à-dire contiennent exactement les mêmes clés avec les mêmes valeurs) et `False` sinon.
Par exemple, `egaux({'e': 3, 'p': 1, 'l': 1 }, {'p': 1, 'e': 2, 'l': 2 })` renvoie `False`.
⚠ On s'interdit ici d'utiliser le test d'égalité `==` entre deux dictionnaires et on écrira un parcours de dictionnaire.

Q7– Écrire une fonction `anagrammes_dico` qui prend en argument deux chaînes de caractères et renvoie `True` si ce sont des anagrammes et `False` sinon.

❑ **Exercice 2** : *Requêtes SQL sur une seule table*
On considère la base de données `pays_du_monde` contenant une seule table `pays` dont le schéma est donné ci-dessous :

pays		
`nom`	:	TEXT
`region`	:	TEXT
`population`	:	INT
`surface`	:	INT
`cotes`	:	INT
`pib`	:	INT

D'autre part, on précise la signification des champs suivants :

— `population` : le nombre d'habitants du pays ;
— `region` : la région du pays (par exemple « Europe de l'ouest ») ;
— `surface` : la surface du pays (en km^2) ;
— `cotes` : la longueur côtière du pays, cette valeur vaut 0 lorsque le pays n'a pas d'ouverture sur la mer ;
— `pib` : le produit intérieur brut par habitant, c'est une mesure de la richesse du pays.

Et on indique que la requête `SELECT DISTINCT region FROM pays` a renvoyé le résultat suivant :

region
Asie
Afrique du nord
Europe de l'est
Europe de l'ouest
Océanie
Afrique subsaharienne
Proche orient
Amérique latine
Amérique du nord

Écrire les requêtes permettant de :

Q8– Trouver la population et le produit intérieur brut de la France.

Q9– Trouver les pays d'Europe n'ayant pas d'ouverture sur la mer.

Q10– Classer par ordre alphabétique les pays d'Amérique latine.

Q11– Lister par ordre décroissant du nombre d'habitants les cinq pays les plus peuplés.

Q12– Trouver le pays du Proche-Orient le plus riche (ayant le `pib` le plus élevé).

Q13– Classer les pays d'Afrique du Nord par densité décroissante de population (la densité est le rapport entre le nombre d'habitants et la surface).

Q14– Classer les régions par somme décroissante du PIB des pays qui les composent.

❑ **Exercice 3** : *Compression par codage par plages (RLE)*
Le **codage par plages** (*Run-Length Encoding* ou RLE) est une méthode classique de compression de données sans perte. Son principe consiste à remplacer les répétitions consécutives d'un même symbole par un couple formé de ce symbole et de son nombre d'apparitions successives.

En Python, le résultat compressé est représenté par une liste de couples `(caractere, compte)`, où `caractere` est de longueur 1 et `compte` est un entier strictement positif.
Par exemple, la chaîne `"AAABBCCCCA"` est encodée par la liste :

```
[('A', 3), ('B', 2), ('C', 4), ('A', 1)]
```

Q15– Donner le résultat du codage RLE de la chaîne `"WWWBWWWWBBBWW"`.
En supposant qu'un caractère et un entier occupent chacun un octet en mémoire, calculer la taille en octets de la chaîne initiale et celle de la liste encodée. Quel est le gain de compression obtenu ?

Q16– Donner un exemple de chaîne pour laquelle le codage RLE conduit à une augmentation de la taille des données (expansion au lieu d'une compression). Quel est le pire des cas pour cet algorithme ?

Q17– Écrire en Python une fonction `rle_encode(texte)` qui prend en argument une chaîne de caractères `texte` et renvoie la liste de tuples `(caractere, compte)` correspondant à son encodage RLE. Si la chaîne est vide, la fonction renverra la liste vide `[]`.

Q18– Écrire en Python la fonction réciproque `rle_decode(liste_rle)` qui prend en argument une liste de couples `(caractere, compte)` et reconstruit la chaîne de caractères initiale.
🛟 On rappelle qu'en Python, l'opérateur `*` entre une chaîne et un entier permet de répéter la chaîne (par exemple `"A" * 3` s'évalue en `"AAA"`).

Q19– On considère la boucle `for car, compte in liste_rle` de la fonction `rle_decode`. Donner un invariant de boucle permettant de prouver la correction partielle de cette fonction, et justifier brièvement pourquoi la terminaison est assurée.

Q20– Écrire une fonction `longueur_totale(liste_rle)` qui calcule et renvoie la longueur totale de la chaîne décompressée sans reconstruire la chaîne en mémoire.

Q21– Déterminer la complexité temporelle de la fonction `rle_encode(texte)` en fonction de la longueur N de la chaîne `texte`. Justifier la réponse.